ARMIS Platform

New Developer Onboarding Guide

Welcome to the Team

This guide walks you through your first week as an ARMIS backend developer. Follow the steps in order. By the end of Day 3, you should have the platform running locally and understand the codebase well enough to pick up a JIRA ticket.

Day 1 — Environment Setup

Step 1: Access & Accounts

Before you start coding, make sure you have access to:

- GitLab (https://gitlab.defencetech.com.tr) — your team lead will add you to the armis group
- JIRA (https://jira.defencetech.com.tr) — request access from canan.arslan@defencetech.com.tr
- Confluence (https://confluence.defencetech.com.tr) — same request as JIRA
- Slack — #armis-dev and #armis-alerts channels
- VPN credentials — required for access to staging/production environments (IT will send separately)

Step 2: Clone & Build

Follow the README.md in the repository root for cloning, environment variable setup, and initial build. The README is the authoritative source for local setup commands.

After setup, run the full test suite to verify your environment:

```
mvn test -pl auth-service,asset-service,mission-service
```

All tests should pass. If any fail on a fresh clone, ping #armis-dev — it may be a known infrastructure issue.

Step 3: TimescaleDB Extension

asset-service uses TimescaleDB for location history. After running the standard migrations, you must enable the extension manually:

```
docker exec -it armis-postgres psql -U armis_user -d armis_db
CREATE EXTENSION IF NOT EXISTS timescaledb;
\q
```

Then re-run the asset-service migration:

```
./scripts/migrate.sh asset-service
```

If you skip this step, asset-service will fail to start with: ERROR: extension timescaledb does not exist.

Day 2 — Codebase Orientation

How to Find Code for a Feature

Follow this approach when you receive a new ticket:

1. Read the ticket description and identify which domain it belongs to (asset, mission, auth, report)
2. Go to the corresponding service module
3. Start from the Controller layer (src/main/java/.../controller/) — this maps HTTP endpoints to use cases
4. Follow the call chain: Controller -> Service -> Repository
5. Domain models are in the domain/ package; DTOs (request/response objects) are in the dto/ package

Package Structure (per service)

```
auth-service/src/main/java/com/defencetech/armis/auth/
  controller/     # REST controllers (@RestController)
  service/        # Business logic (@Service)
  repository/     # JPA repositories (@Repository)
  domain/         # JPA entities (@Entity)
  dto/            # Request and response DTOs
  config/         # Spring configuration classes
  exception/      # Custom exceptions and global handler
```

Where Is the Auth Logic?

Authentication and authorization are handled in auth-service. Key classes:

- JwtTokenProvider — generates and validates JWT tokens
- AuthController — /auth/login, /auth/refresh, /auth/validate endpoints
- SecurityConfig — Spring Security filter chain configuration
- UserDetailsServiceImpl — loads user+roles from DB for Spring Security

The API Gateway calls /auth/validate on every inbound request using a GlobalFilter. This filter is in api-gateway/src/main/java/.../filter/AuthValidationFilter.java.

Where Is the Payment Module?

ARMIS does not have a payment module. It is a mission and asset management platform for defense customers. There are no financial transactions. If you received a ticket mentioning payment, it may be referring to a different project — verify with your team lead.

Day 3 — Making Your First Change

Branching Strategy

- Main branch: main (protected, requires MR + 2 approvals)
- Development branch: develop (protected, requires MR + 1 approval)
- Feature branches: feature/ARMIS-{ticket-number}-short-description
- Bugfix branches: bugfix/ARMIS-{ticket-number}-short-description

Never commit directly to main or develop.

Running Tests

Before pushing, always run tests for the module you changed:

```
mvn test -pl mission-service
```

Integration tests (requires running infra containers):

```
mvn verify -pl mission-service -P integration
```

Opening a Merge Request

- Target branch: develop (never main directly)
- Fill in the MR template (it auto-populates from .gitlab/merge_request_templates/)
- Link the JIRA ticket in the description: Closes ARMIS-{number}
- Assign at least one reviewer from your team

Frequently Asked Questions

Why does the API return 403 on my local setup?

Your test user probably has ROLE_VIEWER. Use the seed admin account for local testing: username admin_local, password defined in scripts/seed-local.sql. See API_DOCS.md for role requirements per endpoint.

How do I add a new endpoint?

1. Add the route in the appropriate Controller class. 2. Add business logic in the Service class. 3. If new DB access is needed, add a Repository method. 4. Add the route to the API Gateway's application.yml routing config. 5. Write a unit test for the Service and an integration test for the Controller. 6. Update API_DOCS.md.

How does inter-service communication work locally?

Services call each other via HTTP REST on localhost using hardcoded ports (defined in each service's application-local.yml). In production, they use gRPC over Kubernetes internal DNS (armis-auth-service.armis.svc.cluster.local). Do not use Kubernetes DNS in local development — it won't resolve.

Onboarding Checklist

Complete the following by end of Week 1:

- GitLab, JIRA, Confluence, Slack access confirmed
- Platform running locally — all health endpoints return UP
- TimescaleDB extension installed
- Full test suite passing locally
- Read ARCHITECTURE.md and ADR.md
- Attended team standup (daily, 09:30 Slack huddle in #armis-dev)
- Introduced yourself in #armis-dev
- Picked up your first ticket (labeled good-first-issue in JIRA)